



LIBRARY MANAGEMENT SYSTEM



by N.VARSHINI AND MANOJNA
Sai University |2026



Introduction:

- In the digital era, libraries are no longer limited to physical books
- Digital libraries store and manage large volumes of data efficiently
- Quick access to information is essential for users
- Manual searching is time-consuming and inefficient
- Data structures help in organizing and retrieving data effectively
- Binary Search Tree (BST) is used for **fast search, insertion, and deletion**
- Genres in the library are stored in **sorted (alphabetical) order**
- This project demonstrates how BST can be applied in a **real-world system**

1

Problem Statement

- Need to manage genres efficiently
- Operations required:
 - Insert
 - Delete
 - Search
 - Update
- Solution: Binary Search Tree

2

What is a BST?

- Binary Search Tree is a hierarchical data structure
- Each node has at most two children
- Property:
 - Left subtree → smaller values
 - Right subtree → larger values
- Enables faster searching than linear methods

Node Structure

- Each node represents a genre
- Contains:
 - Genre name (string)
 - Left pointer
 - Right pointer
- Forms the building block of the tree

3



BST Properties

- Left subtree $<$ Root
- Right subtree $>$ Root
- No duplicate values
- Recursive structure

Node Structure

- Each node contains:
 - Genre (string)
 - Left pointer
 - Right pointer
- Forms the tree

A blue textured background with a large black number 4.

createNode() Function

- Allocates memory using `malloc()`
- Initializes node values
- Returns new node

insert() Function

- Inserts new genre
- Uses `strcmp()` for comparison
- Maintains BST property
- Recursive implementation

5

search() Function

- Searches for a genre
- Traverses left or right
- Efficient search method
- Time complexity: $O(\log n)$

update() Function

- Finds the old genre
- Replaces it with new genre
- May break BST structure

6

deleteNode() Function

- Deletes a node
- Maintains BST rules
- Recursive deletion

Deletion Cases

- Case 1: No child (leaf node)
- Case 2: One child
- Case 3: Two children
- Uses inorder successor

A large, black, stylized number '7' is positioned on the left side of a rectangular area with a blue, marbled, or textured background. The texture consists of swirling, vein-like patterns in various shades of blue and white.

findMin() Function

- Finds smallest element
- Traverses left subtree
- Used in deletion process

display() Function

- Uses inorder traversal
- Prints sorted output
- Left → Root → Right

Algorithm Flow

1. Start program
2. Display menu
3. Perform selected operation
4. Repeat until exit

Sample Input

- Add: Fiction
- Add: Horror
- Add: Science
- Search: Horror
- Update: Horror → Thriller

Advantages

- Fast operations
- Maintains sorted order
- Easy implementation
- Efficient for searching

Limitations

- Can become unbalanced
- Worst case: $O(n)$
- Update method not ideal
- Needs balancing techniques

Output

10

--- Digital Library Menu ---

1. Add Node
2. Delete Node
3. Update Node
4. Search
5. Display
6. Exit

Enter choice: 1

Enter genre to add: Fiction

Enter choice: 1

Enter genre to add: Horror

Enter choice: 1

Enter genre to add: Science

Enter choice: 5

Library Genres:

Fiction

Horror

Science

Enter choice: 4

Enter genre to search: Horror

Genre found!

Enter choice: 3

Enter old genre: Horror

Enter new genre: Thriller

Genre updated successfully!

Enter choice: 5

Library Genres:

Fiction

Science

Thriller

Enter choice: 2

Enter genre to delete: Fiction

Enter choice: 5

Library Genres:

Science

Thriller

Enter choice: 6

Conclusion

- The project demonstrates how **Binary Search Tree (BST)** can be used to manage a digital library efficiently
- BST allows **fast searching, insertion, and deletion** of genres
- Data is maintained in **sorted (alphabetical) order automatically**
- The menu-driven program provides a **simple and user-friendly interface**
- Compared to linear structures, BST improves **performance and efficiency**
- However, BST may become **unbalanced**, affecting performance in worst cases
- This limitation can be solved using **balanced trees (AVL, Red-Black Tree)**



Thank you